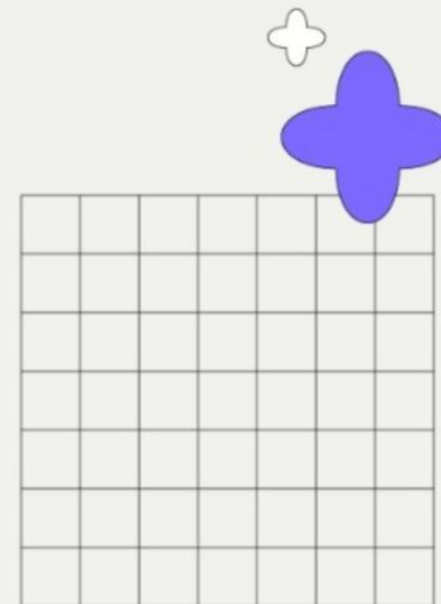
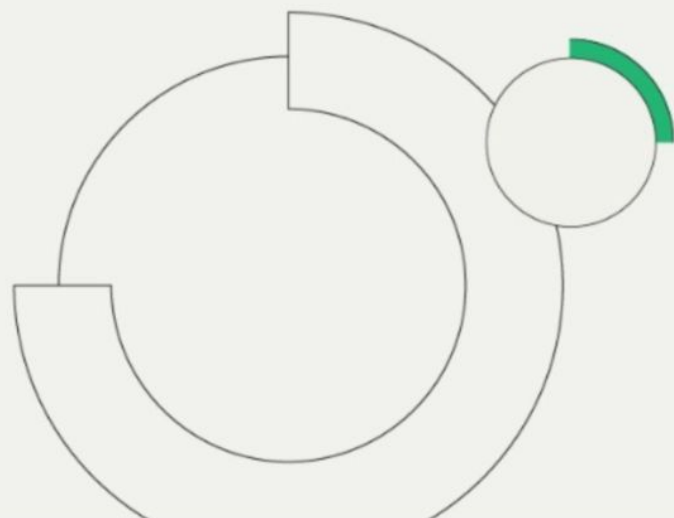




שיעור 9 - יצירת PRD ו MLP



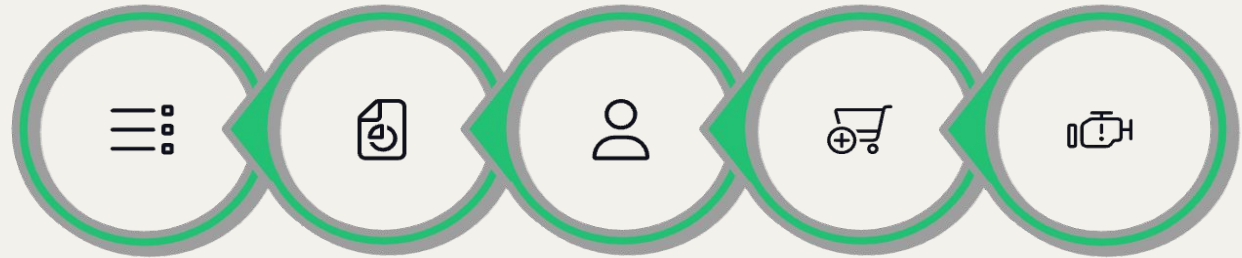
מפגש 9: חלק ראשון מהתוצרים אל PRD מתפתח

קורס מיישם AI

פרויקט מתגלגל: מערכת דיווח תקלות ללא פחד

חברת A B TESTING, מפעל תעשייתי, 400 עובדי שטח

איפה אנחנו בתהליך



דרישות

מדדים

משתמשים
ובעלי עניין

מחקר שוק

בעיה

מה קיים במסמכי הפרויקט שלנו:

- הגדרת הבעיה- [שני וקטורי הפחד (חשש מפגיעה אישית, חרדת החלפה)]
- מחקר שוק והפערים: אף מערכת קיימת לא מטפלת בהתנגדות האנושית לדווח
- דרישות: ארבע דרישות צורך וארבע דרישות רצון
- שמונה סיפורי משתמש מלאים, אחד לכל דרישה
- OKR, KPI ותחשיב ROI

היום נאחד את כל התוצרים האלה למסמך PRD.

מה זה PRD ולמה הוא הכרחי

Product Requirements Document (PRD) הוא מסמך הדרישות שממנו בונים את המוצר.

מה בונים

פתרון לבעיה, לשורש הבעיה
ולערך הנדרש

למי בונים

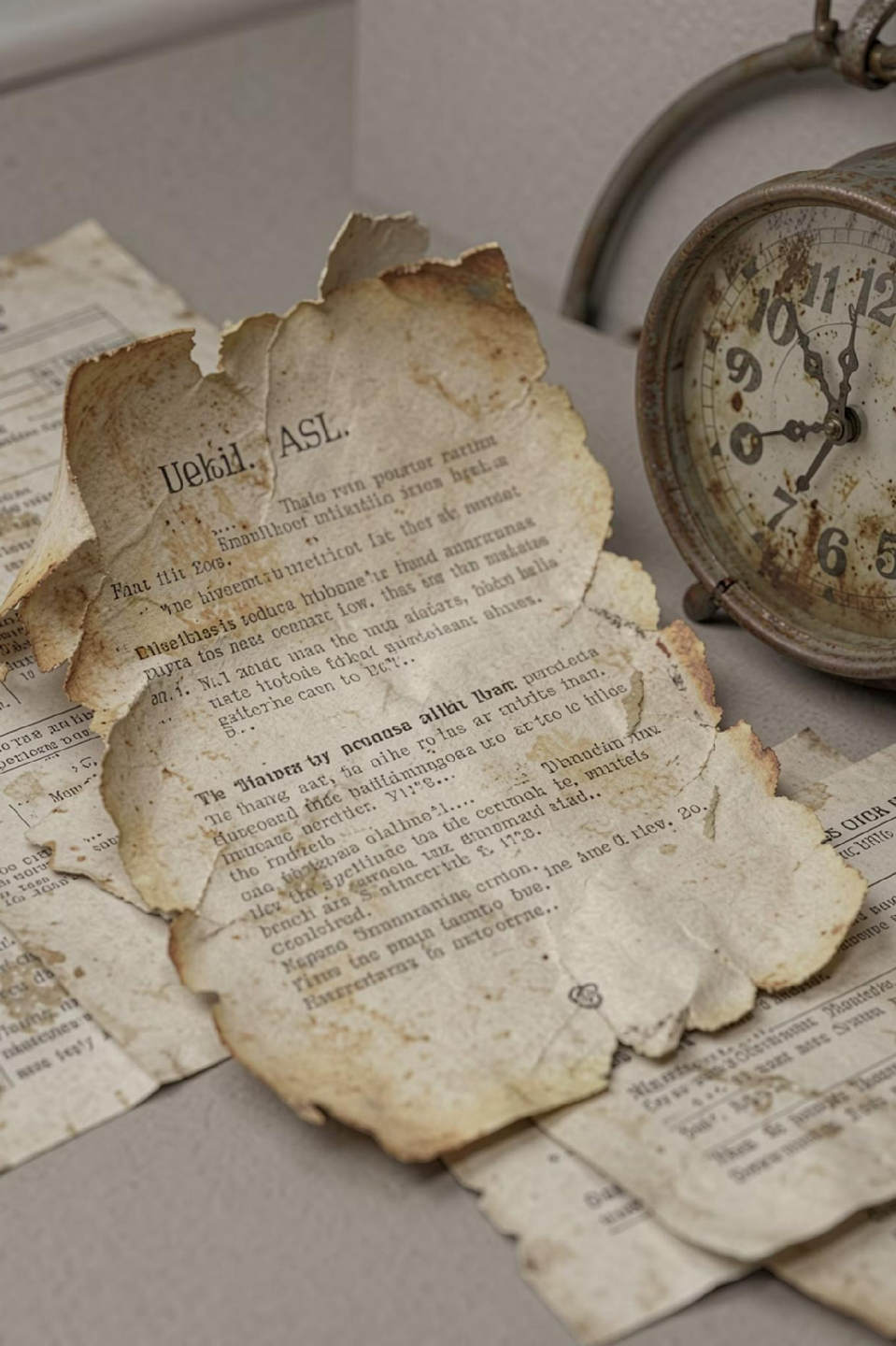
למשתמשים שהגדרנו-
משתמשי צורך ומשתמשי רצון

איך יודעים שהצלחנו

מדדים, קריטריוני קבלה,
אילוצים וגבולות

ה-PRD הוא החוזה בין כל בעלי העניין בפרויקט.

הנהלה, משתמשים, ספקים, מפתחים... כשמחלוקת צצה, חוזרים אליו ואם נדרש מתקנים ומשנים אותו!



הבעיה עם PRD מסורתי

המסמך נכתב פעם אחת בתחילת הפרויקט,
ואז עוצרים

ואז המציאות משתנה:

← החלטות מתעדכנות במיילים וסיכומי ישיבות
החלטות חדשות מתקבלות בישיבות ומועברות במיילים

← דרישות משתנות
דרישות משתנות והמסמך מתיישן והופך לא רלוונטי

← הזיכרון מחליף את המסמך
אחרי חודש, בעלי העניין מסתמכים על הזיכרון

התוצאה: המסמך הרשמי והפרויקט חיים בשני עולמות נפרדים.

העיקרון: PRD מתפתח

ה-PRD הוא מסמך שמתפתח עם הפרויקט. נעדיף לנהל אותו בצ'ט אחד קבוע, מהיום הראשון ועד ההשקה.

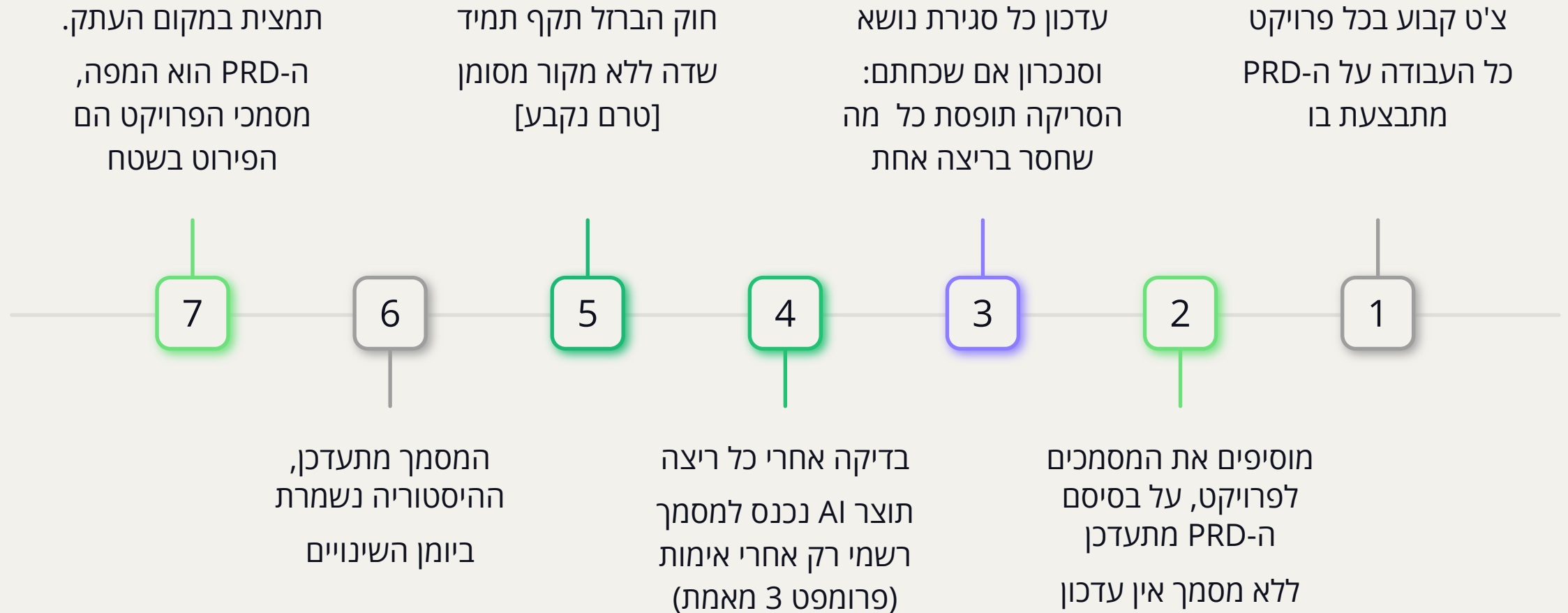
שלושה פרומפטים מפעילים את המנגנון:

3	2	1
פרומפט הבדיקה רץ אחרי כל הקמה או עדכון, מוודא שאין המצאות	פרומפט העדכון רץ בכל פעם שנושא נסגר ותוצר חדש נשמר בפרויקט	פרומפט ההקמה רץ פעם אחת בתחילת הפרויקט, יוצר את המסמך וממלא את מה שכבר קיים

זהו כלי עבודה מקצועי שילווה אתכם בכל פרויקט: בקורס, בארגון, ואצל לקוחות.

[מנגנון-PRD-מתפתח-כלי-עבודה-למיישם-AI.md](#)

שבעת כללי הריצה



חוק הברזל: אפס המצאות

כל שדה שאין לו מקור במסמכי הפרויקט מסומן **[טרם נקבע]**.



בפרויקט אמיתי, שדה מומצא ב-PRD הוא החלטה עסקית שאיש מעולם לא קיבל או מומצא.

שלוש רמות ודאות לכל נתון:

[מאומת]	[מסתבר]	[טרם נקבע]
יש מקור ישיר במסמכים	הנחה שסומנה ככזו וממתינה לאימות	אין מקור, השדה ממתין להחלטה

דוגמה מהפרויקט שלנו: עלות שעת השבתה סומנה **[לא ידוע, נדרש נתון מהלקוח]**, והדשבורד ממתין לה.

מבנה ה-PRD: תשעה חלקים

חלק א בסיס ואסטרטגיה בעיה, ערך, שוק, משתמשים, ROI, מדדים, דרישות, סיפורי משתמש	חלק ב גבולות הגרסה ואב-טיפוס קו ה-MLP, אב טיפוס, אימות מול משתמשים	חלק ג ארכיטקטורת מערכת מקרי שימוש, תהליכים, ארכיטקטורה, מסכים	חלק ד בנייה ואינטגרציות פרומפטים, סוכן, RAG, אוטומציות
חלק ה איכות ואבטחה בדיקות, אבטחה, בקרה אנושית	חלק ו דרישות לא פונקציונליות	חלק ז תפעול ומעבר לייצור	חלק ח היערכות להשקה
חלק ט סיכונים ולוח זמנים			

מסיפורי המשתמש נבנה ה-PRD, ממנו בונים את אב הטיפוס. אחרי אישורו מתקבלות החלטות הפיתוח.



ההדגמה: ריצת הבדיקה על מערכת התקלות

הרצנו את המנגנון על הפרויקט המתגלגל. זו התוצאה:

20

סעיפים ממתינים
לשיעורים הבאים: מסכים, מקרי
שימוש, ארכיטקטורה, פרומפטים,
בדיקות, אבטחה

11

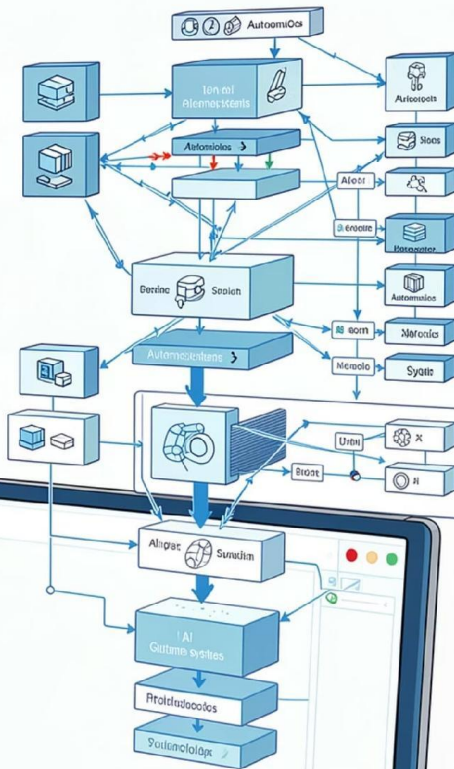
סעיפים מולאו
מתוך מסמכי הפרויקט הקיימים:
הבעיה, הערך, השוק, המשתמשים,
המדדים, הדרישות ושמונת הסיפורים

0.2

גרסה נוכחית
יומן השינויים תיעד כל ריצה: גרסה
0.1 בהקמה, גרסה 0.2 אחרי שילוב
הסיפורים

שדות ללא מקור סומנו: מוביל הפרויקט, לוח זמנים, תיעדוף MoSCoW.

המסמך אמין כי הוא יודע לדווח מה חסר בו.



מבט אל הסעיפים בהמשך: ארכיטקטורה, פרומפטים מערכתיים, בדיקות. כדי למלא אותם, נדרשת חשיבה מותאמת AI.

עד היום אנחנו שואלים, המודל עונה, אנחנו מתקנים.

אנחנו השכבה החכמה.

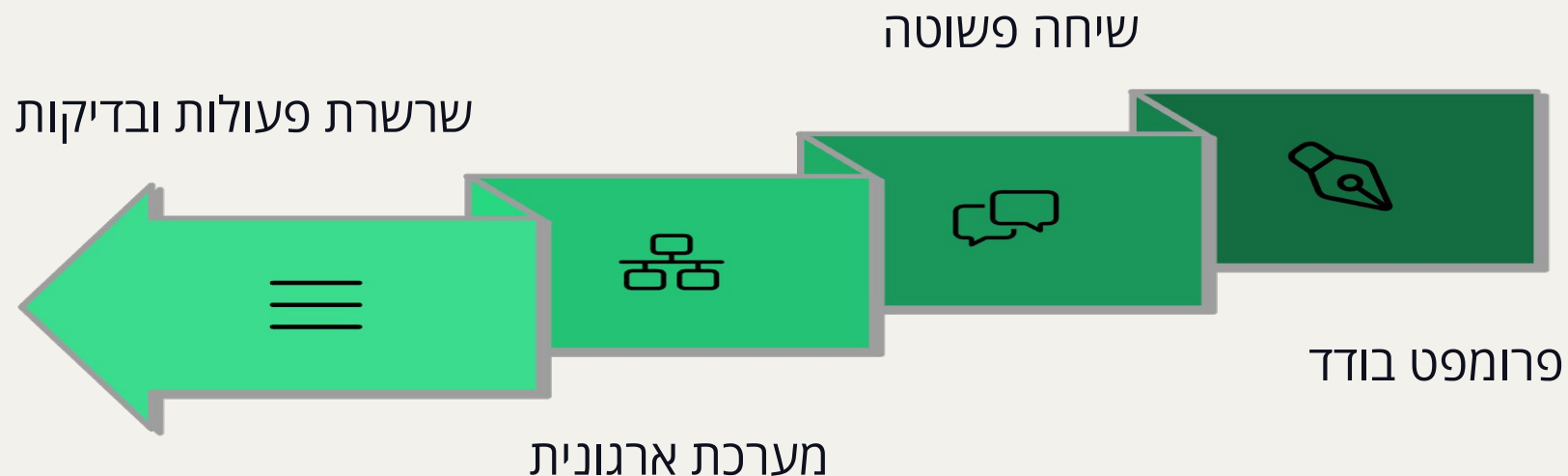
מערכת אוטונומית: בהמשך

במערכת התקלות אין אדם באמצע: הודעת וואטסאפ נכנסת בשתיים בלילה, והמערכת צריכה להבין, לסווג, ולפעול לבד.

מושג 1: פרומפט בודד לעומת מערכת אוטונומית

פרומפט בודד עובד מצוין בשיחה.

מערכת ארגונית דורשת חשיבה ארכיטקטונית: שרשרת של קריאות קטנות וממוקדות שמבצעות פעולות, שכל אחת ניתנת לבדיקה.



מושג 2: שלוש השכבות

כל מערכת AI בנויה משלוש שכבות:
קלט ← עיבוד ← פלט.

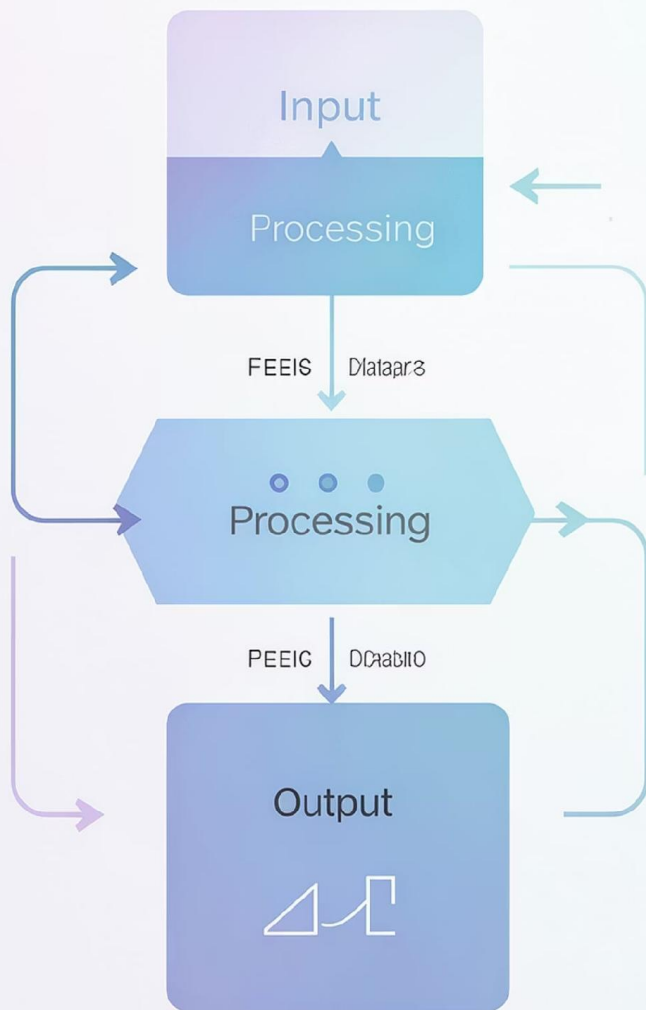
מה מייצרים:

הודעה קולית, נתונים או צילום מהשטח (קלט) ←

ה-AI מנסח ומסווג (עיבוד) ←

רשומת תקלה מובנית נכנסת למערכת (פלט).

יורחב במפגש 12.



מושג 3: פלט מובנה

בצ'אט, הפלט הוא טקסט חופשי שאדם קורא. במערכת, תוכנה צריכה לקרוא את הפלט, ולכן הוא חייב להיות עם מבנה נתונים קשיח.

דוגמה: רשומת דיווח במערכת התקלות:

```
line: "קו 3"  
severity: "בינונית"  
category: "בטיחות"  
description: "שמן על הרצפה ליד המכש"
```

שימו לב למה שחסר במבנה: שדה מזהה ייחודי. הפרדת המדווח מהתוכן קורית כבר ברמת מבנה הנתונים. על זה נרחיב במפגש 13.



מושג 4: נתיב מילוט

מערכת טובה יודעת מתי היא לא בטוחה, ואז מעבירה לאדם יחד עם כל ההקשר שנאסף.

סיווג ברמת ביטחון נמוכה
עובר לבחינת ממונה הבטיחות

דיווח על אירוע חמור
מפעיל את החריג הרגולטורי המוגדר בנוהל

זה עיצוב נכון, והוא יהפוך לסעיף הבקרה האנושית ב-PRD.
יורחב במפגש 12.

הנחיות ליצירת PRD: נקים את ה-PRD של הפרויקט שלנו

1

ודאו שכל תוצרי המפגשים
הקודמים שמורים
כל תוצרי המפגשים הקודמים
שמורים במסמכי הפרויקט

2

פתחו צ'ט חדש בתוך הפרויקט
קראו לו: **PRD**, **מסמך חי**

3

הריצו את פרומפט ההקמה
מהמסמך "מנגנון ה-PRD החי"

4

הריצו את פרומפט הבדיקה
על התוצאה

5

עברו על המסמך
האם כל סעיף ממולא נשען על מסמך אמיתי? האם
כל [טרם נקבע] מוצדק?



אם המודל נעצר באמצע, כתבו: **המשך מהסעיף שבו עצרת.**

עכשיו תורכם

הריצו את הפרומט הראשון והשלישי ליצירת PRD לפרויקט שלכם



יצרנו את ה-PRD

חלק א של ה-PRD שלכם מלא:

1. הבעיה
2. הערך/הבזבוז
3. המשתמשים
4. המזדים
5. הדרישות וסיפורי המשתמש.

זה הרגע שבו רוב הפרויקטים עושים את הטעות: קופצים ישר לפיתוח.

אנחנו נעשה אחרת: נייצר את ה-MLP.



חלק שני - יצירת ה-MLP

Minimum Lovable Product

קורס מיישם AI, הפרויקט המתגלגל: מערכת דיווח
תקלות ללא פחד



מה זה MLP

:MLP (Minimum Lovable Product)

הגרסה הראשונה של המערכת.

- עונה
 - על כל דרישות הצורך
 - על מינימום שכבת הרצון שהופך אותה לאהובה
- משמשת אב טיפוס: מציגה למשתמשים מה המערכת עושה
- אם אושרה מול המשתמשים, ממשיכים לפתח את שאר המוצר

הרצון 

המשתמש רוצה
לחזור לדווח.
הדיווח החוזר נוצר
מתוך חוויה חיובית.

הצורך 

המערכת עובדת.
המשתמש מנסה
אותה. דיווח ראשון
ואז... שתיקה.

ה-MLP שלנו 

4 דרישות צורך + דרישת רצון אחת = גרסה
ראשונה אהובה.

מדוע Lovable במקום Viable?

MVP קלאסי עונה על הצורך בלבד: המערכת עובדת, והמשתמש יודע להשתמש בה.

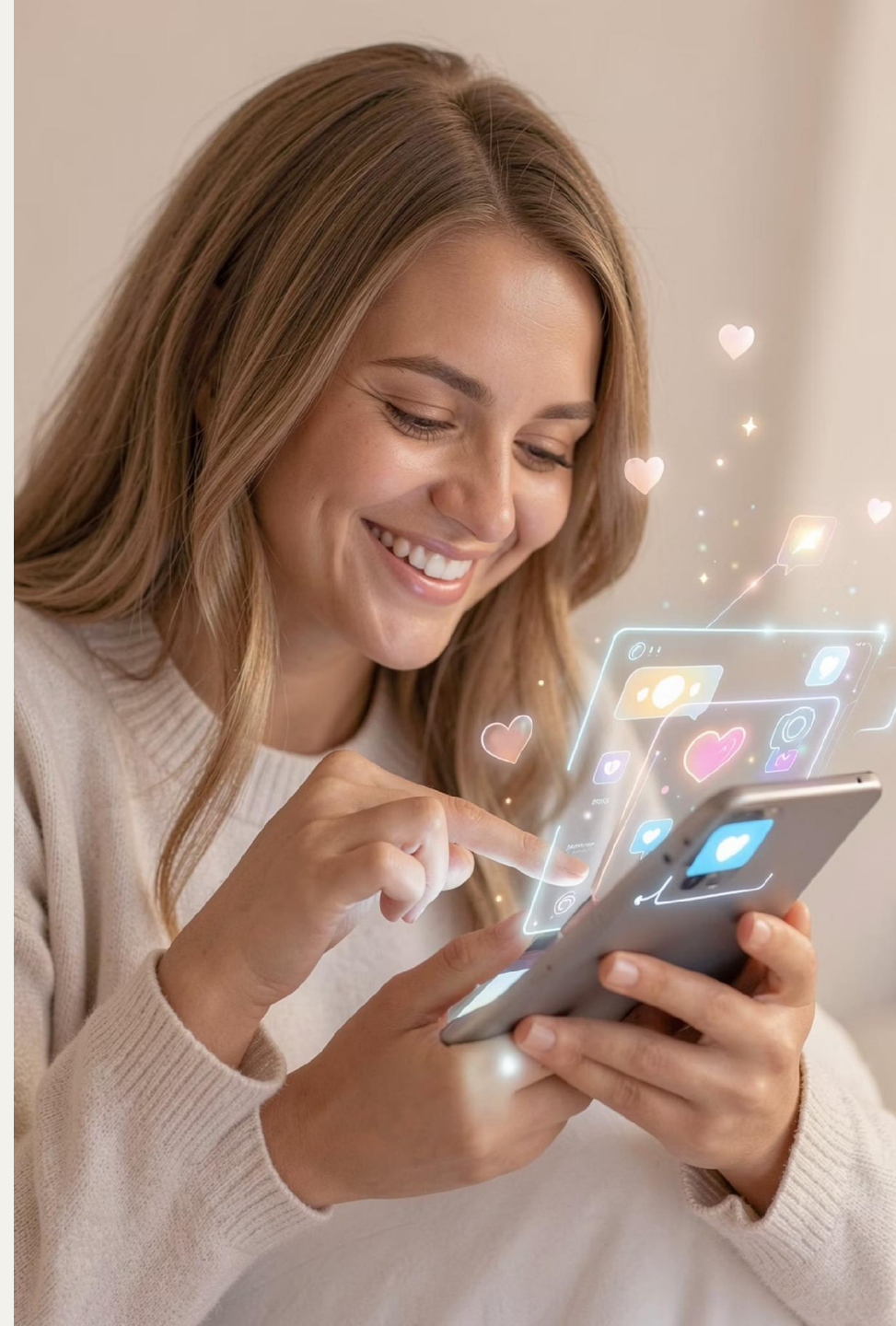
MLP מוסיף את המינימום שגורם למשתמש לרצות לחזור אליה.

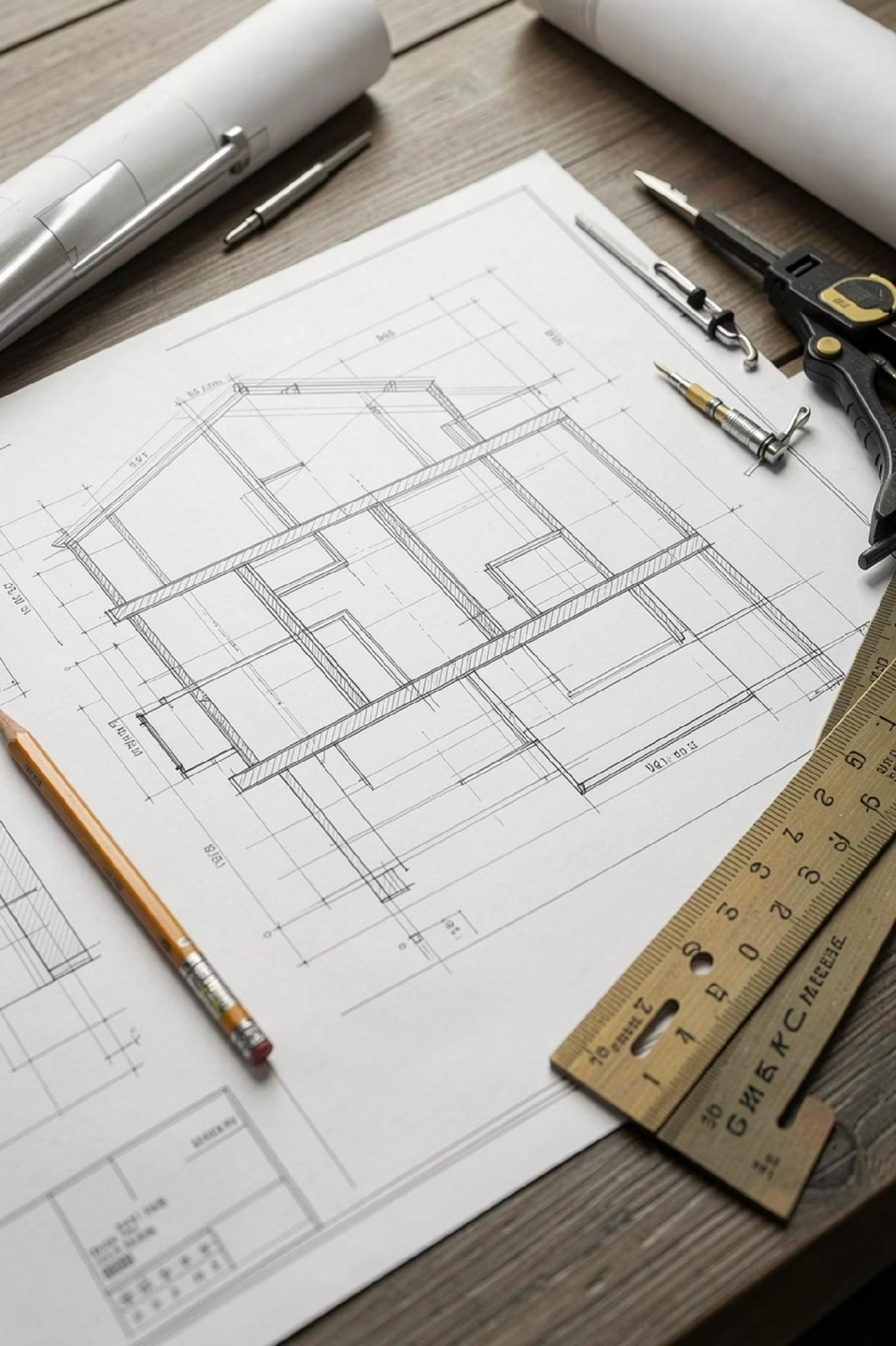
ההבדל הוא קריטי: גרסה ראשונה שעונה רק על הצורך משחזרת בדיוק את שורש הבעיה של וקטור 2

כשמערכת "לוקחת" מהעובד מידע ולא מחזירה לו את מה שהוא רוצה, העובד ידווח פעם אחת ויפסיק.

- הצורך מייצר את הדיווח הראשון
- הרצון מייצר את הדיווח החוזר

בפרויקט שלנו: ארבע דרישות הצורך (אנונימיות, דיווח מהיר, תמונת אמת, סגירת SLA), ועוד דרישת רצון אחת [ממתין לאישור המועמד: קרדיט אישי, "הבעיה טופלה בזכותך"].





יצירת אב טיפוס לפני הפיתוח

עובד שטח במפעל יגיד דברים מדויקים מול מסך דיווח, ויגיד מעט מאוד מול מסמך.

1

עלות הטעות ⌚

אב הטיפוס עולה שעות, וטעות בפיתוח עולה שבועות

2

גילוי החריגים 🔍

החריגים האמיתיים מתגלים מול מסך: "מה קורה כשאין קליטה במרתף?"

3

החלטות הפיתוח 🏗️

רק אחרי האישור, מתחילים פיתוח: מה מפותח ללא AI, היכן נדרש AI ובאיזו דרגת מודל, ממשקים ומבנה מודולרי

וחשוב מכל: אב הטיפוס מציג מה המערכת עושה
הוא לא המוצר המוגמר.

המנגנון: שלושה שלבים, אישור אנושי בין שלב לשלב

הפרומפט המלא נמצא במסמך "מנגנון ה-PRD המתפתח" בפרויקט.

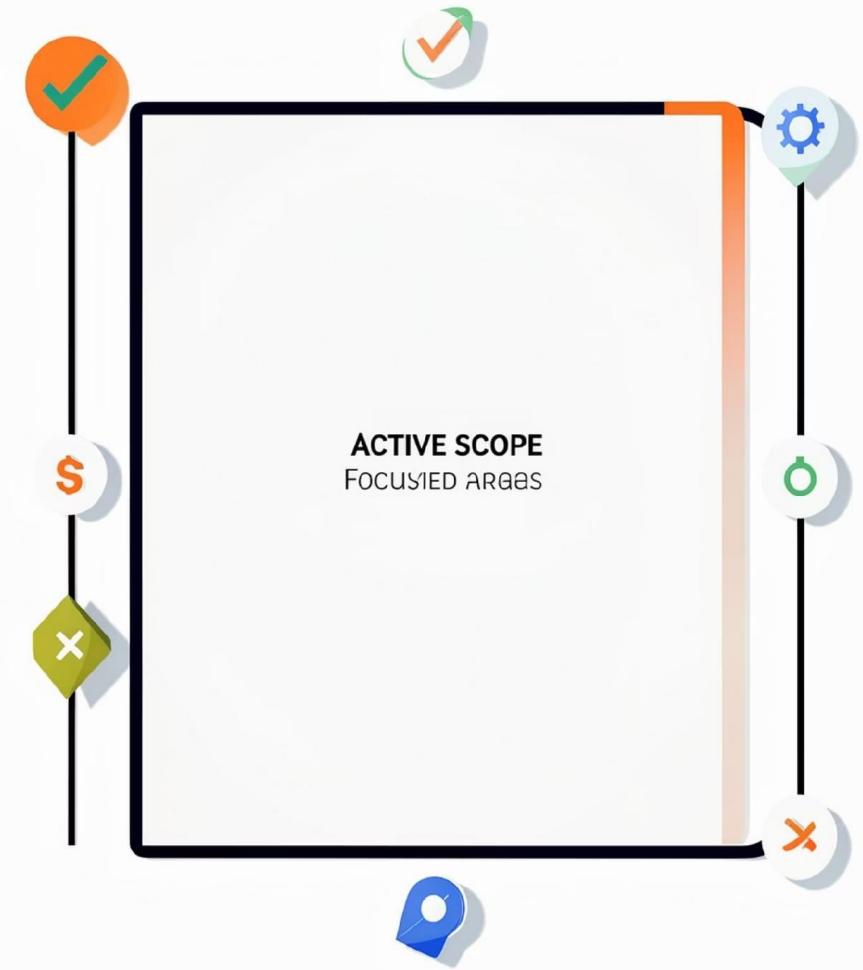
שלב	מה קורה	מי מכריע
א: הגדרת הגבולות	המודל מציע מה נכנס לגרסה הראשונה, מה נדחה ולמה, וגוזר קריטריוני קבלה	אתם מאשרים את הקו
ב: בניית אב הטיפוס	המודל בונה מתוך ה-PRD וסיפורי ה-MLP בלבד	אתם מאשרים את רשימת המסכים לפני הבנייה
ג: לולאת התיקון	ממצאי האימות מתוקנים בעדכון נקודתי, סבב אחרי סבב	המשתמשים קובעים מתי זה מוכן

העיקרון בכל שלושת השלבים: המודל מציע, האדם מכריע.

שלב א: הגדרת גבולות ה-MLP

מה הפרומפט עושה:

- מייצר HTML
- כל ההחלטות שקבלנו ב-PRD מיושמות במסכים
- אנחנו אמורים לבחון את המסכים ולהחליט האם עונים לדרישות או שנדרש שינוי.



שלב ב: בניית אב הטיפוס

המודל בונה מתוך ה- בהתאם לסיפורי המשתמש המורחבים.

הגבולות של אב הטיפוס:



גבולות קפדניים
מה שמחוץ ל-MLP אינו מופיע בו



נתונים סינטיים בלבד
ללא חיבור למערכות אמת, ללא שמירת מידע אישי



שאלת בדיקה לכל מסך
כל מסך מלווה בשאלה שהוא בודק מול המשתמש



סימון בינה מלאכותית
היכן שתפעל בינה מלאכותית (ניסוח, סיווג), מוצגת תוצאה
מדומה עם סימון: "כאן תפעל בינה מלאכותית"

שלב ג: לולאת התיקון

אימות מול משתמשים כמעט תמיד מחזיר ממצאים. זה סימן שהאימות עבד.

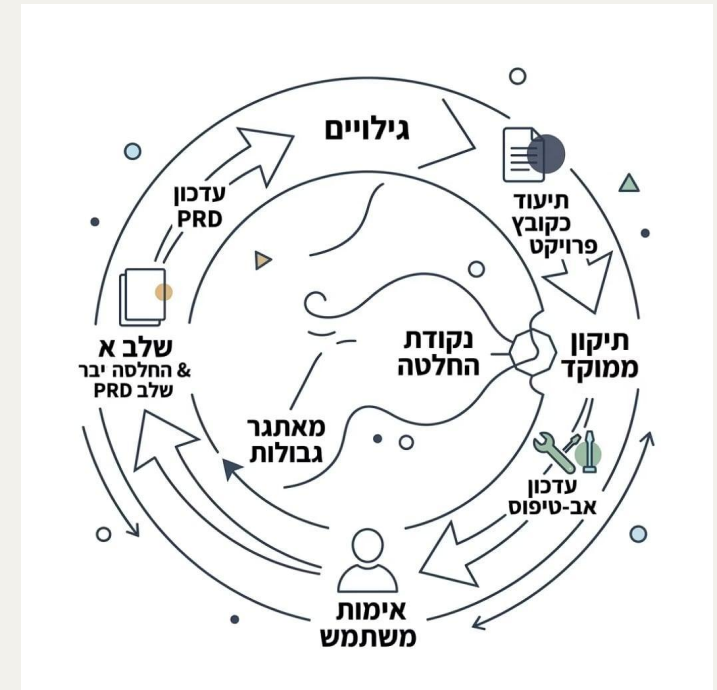
כלל ראשון: תיעוד לפני תיקון

הממצאים נשמרים כמסמך בפרויקט. ללא מסמך לא יתבצע שום תיקון בריצת התיקון.

כלל שני: לאן חוזרים?

- **ממצא בתוך הגבולות** (זרימה, מסך, ניסוח): תיקון נקודתי בשלב ג. אב הטיפוס מתעדכן, ואינו נבנה מחדש
- **ממצא שמערער על הגבולות** (דרישה שצריכה להיכנס או לצאת): חזרה לשלב א, הכרעה מחודשת שלכם, ועדכון מסמכים רלוונטיים וה- PRD

תנאי היציאה מהלולאה: אב הטיפוס עומד בתנאי האישור שהוגדרו בשלב א.



תרגול סיום: נריץ את שלב א של החלק הרביעי עכשיו

התוצר של כל קבוצה בסוף השיעור: גבולות MLP מאושרים, שמורים בפרויקט, ומשוקפים בסעיף 12 של ה-PRD.

הריצו על הפרויקט שלכם

הפנו למסמך ולפרומט הרלוונטי
אין צורך להעתיק כלום!!!

ודאו שה-PRD שמור
ודאו שה-PRD שהקמתם בתרגול
הקודם שמור, וחלק א שלו מלא

אשרו ושמרו
אשרו או תקנו, שמרו את התוצר
בפרויקט, והריצו עדכון בצ'ט
ה-PRD

בחנו את ההצעה
האם התוצאים נכונים? האם
דרישת הרצון שנבחרה באמת
מייצרת שימוש חוזר?

עכשיו תורכם

צרו את מסכי הפרויקט שלכם - את MLP



סיכום השיעור

מה יש לנו היום שהיה חסר הבוקר:

PRD מתפתח

מסמך אחד שמרכז את כל
התובנות, עם צ'ט קבוע שמנהל
אותו

גבולות MLP מאושרים

הקו בין הגרסה הראשונה לשלב
הצמיחה, כהחלטה מתועדת

מנגנון שממשיך איתנו

כל תוצר עתידי נשמר בפרויקט ומתעדכן ב-PRD בריצה אחת

הצעד הבא: שלב ב, בניית אב הטיפוס, מתוך ה-PRD שלכם. ואחרי
האישור מגיעות החלטות הפיתוח: מקרי השימוש, הארכיטקטורה, והבנייה
עצמה.



שיעורי בית ומבט קדימה

שיעורי הבית

סיום יצירת PRD

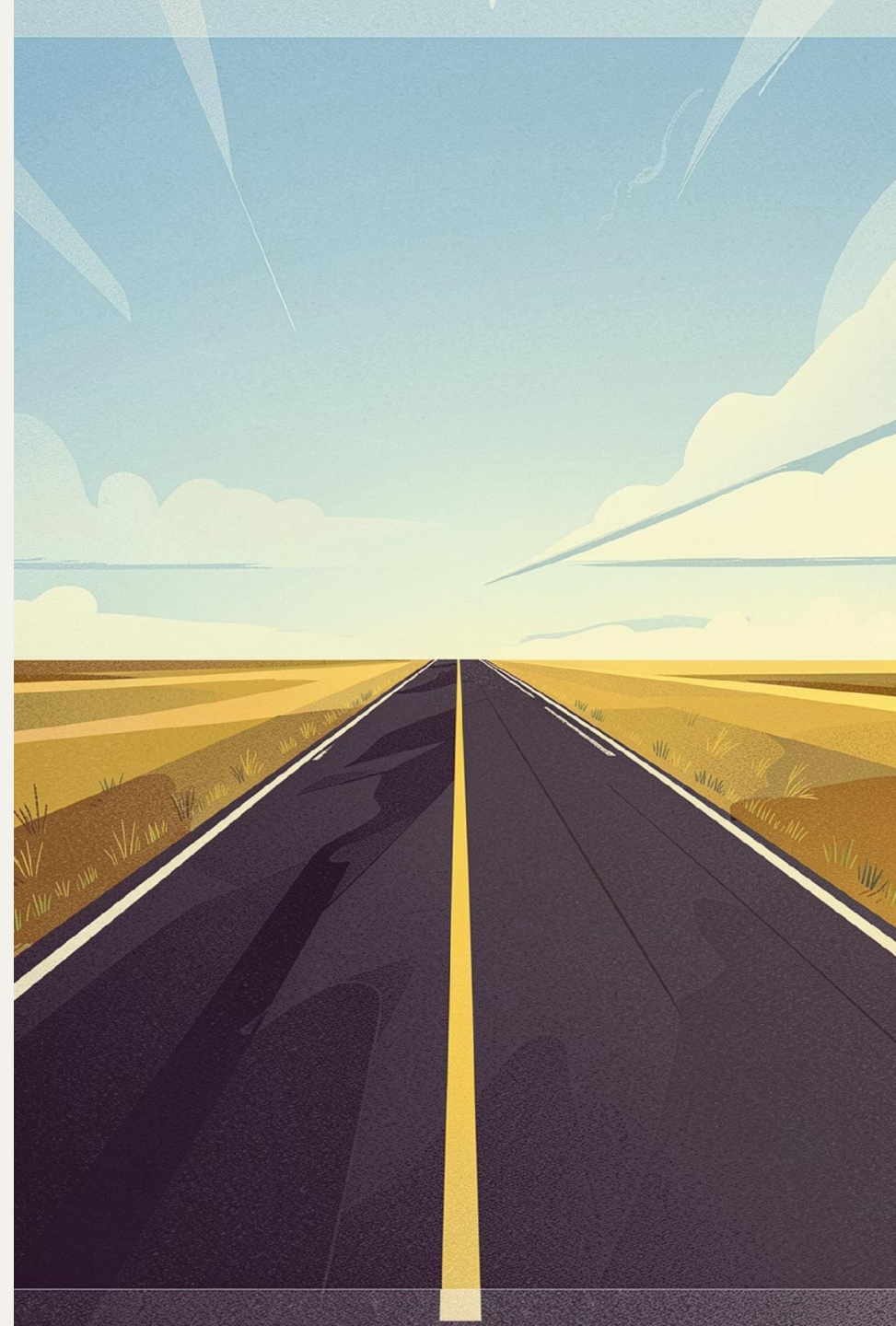
סיום יצירת מסכי המערכת

MLP

מפגש 10: מה בהמשך?

מסיפורי המשתמש אל Use Cases ותהליכים.

התהליכים שנכתוב שם
יהפכו בהמשך תהליכי בניית
המערכת
התרחישים שהכנסנו לסיפורי
המשתמש ישמשו לכתיבת
הבדיקות



רטרוספקטיבה

שלוש שאלות לסיום

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו - תפיסה
שהתהפכה, הנחה שנשברה.

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.



תודה רבה- זכרו

“

עיקרון ראשון
"ככל שקל יותר לבנות, ההחלטה האם בונים
הופכת קריטית יותר."

”

“

עיקרון שני
"בודקים את כל העלויות ולא רק את עלויות הפיתוח"

”

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

